FlowCabal 架构设计文档 v6

日期：2026.02.27

AI 辅助长篇写作的 DAG 工作流引擎—“文本版 ComfyUI”。

目录

FlowCabal 架构设计文档 v6 1
问题与动机 3
  长篇小说写作的核心困难 3
  设计演化 3
    v4：正确但过重 3
    v5：最小可用 3
    v6：状态中心化（当前） 3
  保留的核心洞察 4
  已废弃的概念 4
整体架构 4
  单进程 TypeScript 4
  Monorepo 结构 4
核心概念 5
  三层解耦：Workflow / Workspace / Project 5
  用户偏好 6
  TextBlock 6
  DAG 连接 6
类型系统 6
  核心类型 6
  LLM 配置 7
  多版本缓存 7
  per-node 覆盖 8
  Workflow 导入/导出 8
  类型与 Schema 的关系 8
记忆架构 9
  设计哲学 9
    约束查询是核心能力 9
    Memory 是 manuscripts 的有损缓存 9
    不用 RAG 9
  种子文件 9
    为什么不是更多种子文件 9
    生成性事实 vs 派生断言 10
  三级上下文加载 10
  Memory 文件格式 10
  人和 Agent 共同读写 10
Runner-Core：增量构建引擎 10
  执行模型 10
    失效级联 11
    子图执行 11
    Token 估算 11

State 中心化 11
loadState 11
同步读 / 异步写 11
NodeStatus 计算 12
依赖追踪 12
Workspace API 12
查询（同步） 12
Mutation（异步） 12
执行 12
事件与 Agent 13
Run 执行循环 13
启动与时序 13
执行流程 13
LLM 配置解析 13
RunHandle 13
step 模式 13
RunSummary 14
缓存二维失效 14
结构性失效（自动） 14
上下文过期（预警） 14
Workspace 生命周期 14
事件模型 14
RunEvent（执行生命周期） 14
StateEvent（状态变更） 15
双通道的好处 15
Agent 系统 15
单 Agent + Tool Calling 15
两种运行模式 16
runAgent（非流式） 16
conversationalAgent（流式） 16
agent-inject 机制 16
RuntimeContext 16
四种系统提示词 16
目录结构 17
全局配置 ~/.config/flowcabal/ 17
项目本地 <project>/.flowcabal/ 17
LLM 集成 17
Provider 工厂 17
生成接口 18
前端适配 18
架构决策记录 18
为什么不用 RAG 18
为什么纯文件系统 19
为什么 Vercel AI SDK 19
为什么增量构建而非全量执行 19
为什么 agent-inject 只预警不自动失效 19

为什么 State 中心化 19
为什么 llm/ 在顶层而非 runner-core 下 20
为什么 types.ts 和 schema.ts 分开维护 20
实施路线 20
Phase 1: Headless Engine + CLI（已完成） 20
Phase 2: 可视化 DAG 编辑器（当前） 20
Phase 3: 高级能力（未来） 20

问题与动机

长篇小说写作的核心困难

长篇小说（10 万字+）的 AI 辅助写作面临三个根本性挑战：

1. **上下文窗口不够装**：即使 200K token 的窗口也无法一次性容纳一部完整小说加全部设定。必须有选择地加载上下文
2. **一致性随长度崩溃**：角色性格漂移、设定自相矛盾、伏笔遗忘—这些不是幻觉，是**上下文缺失**的直接后果
3. **生成流程不可控**：端到端的“给我写一章”无法让作者介入。作者需要拆解、调试、迭代每一步

FlowCabal 的核心主张：**人类定义 what（工作流结构、创作意图），AI 负责 how（上下文检索、约束满足、文本生成）。**

设计演化

v4：正确但过重

v4 的架构是“教科书式正确”的全功能设计：向量搜索（OpenViking embedding）、五种侧写类型、Python + Browser + WebSocket 三进程协调、三角色 Agent（A/B/C）。

核心问题：

- **向量搜索是杀鸡用牛刀**：小说约束空间 50-100KB，Agent 读一个索引文件就能决定加载什么，比余弦相似度更准确—Agent 理解叙事上下文
- **三进程 IPC 链路**：Python + Browser + WS 的偶发复杂度远超本质复杂度
- **三角色 Agent 是过早优化**：没有可工作的执行引擎之前，角色分拆是空中楼阁

v5 策略：**先让最小可用版本跑起来**，再逐步添加复杂度。

v5：最小可用

v5 实现了完整的 headless engine：TypeScript 单进程、DAG 拓扑排序、增量构建、Agent tool calling、CLI。

但面向 TUI/GUI 审视时发现两个根本矛盾：

1. **Workspace 无状态**：每次查询走磁盘。10 节点的 getDashboard = 20+ 次 readFile
2. **Run 私有状态**：run.ts 内部的 `outputs: Map` 对外不可见，Agent 执行期间的 query_runtime 完全废掉

v6：状态中心化（当前）

引入 `state.ts` 作为唯一内存模型。workspace 和 run 都读写它：

```
workspace.ts (API)  →  state.ts (truth)  ←  run.ts (execution)
                            ↓
                        cache.ts (I/O)
```

同时修正了 v5 的遗留问题：路径分层（全局 vs 项目本地）、双通道事件模型、多版本缓存、RunHandle.done Promise。

保留的核心洞察

从 v4 及更早版本保留的设计决策：

- **DAG 工作流 + 拓扑排序执行**：核心抽象不变
- **TextBlock**：节点 prompt 的构成单元，通过引用传递输出
- **L0/L1/L2 层次化上下文**：索引 → 记忆文件 → 原稿的渐进加载
- **有界上下文预算**：无论手稿多长，上下文加载有预算上限

已废弃的概念

以下概念在演化中被证明不再需要：

- **冻结节点输出（freeze）**：v5+ 通过多版本缓存 + version switch 代替
- **环形节点**：迭代精修在 Agent 推理内部完成，Kahn 算法天然拒绝环
- **规约/状态分层（prescriptive/descriptive）**：人和 Agent 共同读写所有记忆文件
- **显式 Edge 类型**：从 TextBlock `kind: "ref"` 隐式推导即可
- **Workspace 无状态查询**：被 v6 的内存模型 + 同步查询取代

整体架构

单进程 TypeScript

```
┌──────────────────────────────────────────────────┐
│  Bun 进程                                        │
│                                                  │
│  ┌──────────┐  ┌──────────┐  ┌──────────────┐   │
│  │ CLI/TUI  │  │  Engine  │  │  LLM Provider │  │
│  │ (yargs + │→│  (DAG +  │→│  (Vercel AI   │  │
│  │  clack)  │  │  Agent)  │  │   SDK)        │  │
│  └──────────┘  └──────────┘  └──────────────┘   │
│                     │                            │
│          ┌──────────┴──────────┐                 │
│          │                     │                 │
│  ┌───────▼──────┐  ┌──────────▼──────────────┐  │
│  │ ~/.config/   │  │ <project>/.flowcabal/     │  │
│  │ flowcabal/   │  │ memory/ + runner-cache/   │  │
│  └──────────────┘  └──────────────────────────┘  │
└──────────────────────────────────────────────────┘
```

没有数据库、没有 WebSocket、没有子进程。一个 Bun 进程，读写文件系统。存储分两层：全局配置 + 项目本地状态。

Monorepo 结构

```
flowcabal/
  package.json              # Bun workspace root
  packages/
    engine/                 # 核心无头引擎（零 UI 依赖）
      src/
```

```
      types.ts            # 领域类型（手写）
      schema.ts           # Zod schemas（手写，运行时校验）
      paths.ts            # 路径注册表（全局 + 项目本地）
      id.ts               # nanoid 36位 ID 生成（小写字母+数字）
      llm/                # LLM 集成（共享基础设施）
        provider.ts       # Vercel AI SDK provider 工厂
        generate.ts       # generateText + streamText 封装
      runner-core/        # 执行引擎
        state.ts          # 唯一内存模型（truth）
        workspace.ts      # 薄 API 层，委托 state
        run.ts            # 执行循环，通过 state 读写
        resolve.ts        # TextBlock 解析 + SHA-256 prompt hash
        budget.ts         # token 估算（CJK/英文双计数）
        workflow.ts       # Kahn 拓扑排序 + 子图计算
        cache.ts          # 磁盘 I/O（readJson/writeJson + workspace 生命周期）
        convert.ts        # Workflow ↔ NodeDef[] 互转（ID 重映射）
      agent/              # Agent 子系统
        agent.ts          # tool-use loop（generate + stream 两种模式）
        assembler.ts      # L0 index.md 加载
        memory.ts         # Memory CRUD + index 自动生成
        tools.ts          # Zod tool 定义（含 query_runtime）
        prompts.ts        # 中文系统提示词（4 种角色 + 共享约定）
  cli/                    # TUI
    src/
      index.ts            # bin: flowcabal
      config.ts           # 配置加载
      commands/           # init, add-chapter, status, generate, store
```

Engine 包零 UI 依赖—不 console.log、不假设 terminal。可以被任何前端（TUI、Web、Electron）使用。

核心概念

三层解耦：Workflow / Workspace / Project

概念	存储位置	职责
Workflow	~/.config/flowcabal/workflows/	纯模板/蓝图。只描述节点结构和 prompt 组合方式（TextBlock[]），不含 LLM 配置。用于朋友间分享工作流
Project	<project>/.flowcabal/memory/	小说项目。拥有独立的 Agent 记忆（角色、世界、文体、定稿章节）
Workspace	<project>/.flowcabal/runner-cache/<id>/	Workflow 的一次实例化运行环境。绑定一个 project，存执行缓存。反复调试，删除即释放

- 同一个 Workflow 可以被多个 Workspace 实例化
- 同一个 Project 可以在不同 Workspace 中使用不同的 Workflow

· Workflow 是**分享**的单元，Workspace 是**工作**的单元

用户偏好

per-node LLM 覆盖存在 ~/.config/flowcabal/preferences/<workflow-id>.json，跨工作区生效。解决“每次新建 workspace 都要重配 LLM”的问题，同时不污染 workflow 模板的纯净性。

TextBlock

节点 prompt 的构成单元，三种类型：

```
type TextBlock =
  | { kind: "literal"; content: string }
  | { kind: "ref"; nodeId: string }
  | { kind: "agent-inject"; hint: string };
```

· literal — 静态文本，用户直接编写
· ref — 引用上游节点输出，执行时动态替换为上游的 current version output
· agent-inject — Agent 注入点。hint 告诉 Agent 方向，Agent 读 L0 自主决定注入什么内容。一个节点可以有多个注入点

ref 同时定义了 DAG 的隐式连接关系—不需要显式 Edge 类型。依赖关系通过扫描 systemPrompt + userPrompt 中的 ref 块自动推导。

DAG 连接

没有显式 edges 数据结构。连接关系从 TextBlock 的 kind: "ref" 隐式推导。extractNodeDeps() 扫描每个节点的 systemPrompt 和 userPrompt，构建 Map<nodeId, Set<depIds>>。

这意味着：
· 用户无需手动“连线”—在 prompt 中引用一个节点就自动建立依赖
· 编辑 prompt 的 ref 块就是编辑 DAG 结构
· 依赖图始终与 prompt 内容一致，不会出现“连了线但没引用”的不一致

类型系统

核心类型

```
// ── 节点定义 ──
interface NodeDef {
  id: string;          // nanoid 36位（小写字母+数字）
  label: string;
  systemPrompt: TextBlock[];
  userPrompt: TextBlock[];
}

// ── 工作流（纯模板，无 LLM 配置，无运行状态）──
interface Workflow {
  id: string;
  name: string;
  nodes: NodeDef[];
}

// ── 项目配置 ──
interface ProjectConfig {
```

```
  name: string;
}

// ── 工作区元数据 ──
interface WorkspaceMeta {
  projectId: string;
  createdAt: string;  // ISO 8601
}
```

节点不存储 LLM 配置。LLM 选择在运行时由以下优先级决定：

1. workspace preferences.json 中的 per-node 覆盖
2. ~/.config/flowcabal/llm-configs.json 中的 "default" 配置

LLM 配置

```
type LlmProvider =
  | "openai" | "anthropic" | "google"
  | "mistral" | "xai" | "cohere"
  | "openai-compatible";

interface LlmConfig {
  provider: LlmProvider;
  baseURL?: string;          // openai-compatible 必填
  apiKey: string;
  model: string;
  temperature?: number;      // 0-2
  maxTokens?: number;
  topP?: number;             // 0-1
  frequencyPenalty?: number;  // -2 ~ 2
  presencePenalty?: number;   // -2 ~ 2
  providerOptions?: Record<string, Record<string, JsonValue>>;
}

// llm-configs.json: Record<name, LlmConfig>, "default" 为默认配置
type LlmConfigsFile = Record<string, LlmConfig>;
```

用户在 ~/.config/flowcabal/llm-configs.json 中维护多套配置，按名字索引。openai-compatible 复用 @ai-sdk/openai，覆盖 baseURL，支持 DeepSeek 等第三方 API。

多版本缓存

```
type VersionSource =
  | { kind: "generated" }            // LLM 生成
  | { kind: "human-edit" }           // 人工编辑
  | { kind: "conversation"; summary: string };  // 对话式修改

interface ExecutionTrace {
  model: string;
  inputTokens: number;
  outputTokens: number;
  durationMs: number;
  resolvedSystem: string;     // 实际发给 LLM 的 system prompt
  resolvedUser: string;       // 实际发给 LLM 的 user prompt
}

interface NodeVersion {
  id: string;                              // nanoid
  promptHash: string;                      // SHA-256 (literal+ref 展开, agent-inject
```

```
占位)
  agentInjects: Record<string, string>;    // hint → 注入内容
  output: string;
  source: VersionSource;
  current: boolean;                        // 恰好一个为 true
  createdAt: string;                       // ISO 8601
  trace?: ExecutionTrace;                  // 仅 generated 有
}

// 磁盘格式: outputs/<node-id>.json
interface NodeVersionFile {
  versions: NodeVersion[];
  currentId: string;
}
```

每个节点可以有多个版本。版本切换（pickVersion）只修改 current 标记和 currentId，不删除旧版本。ExecutionTrace 记录了完整的生成上下文—resolvedSystem/resolvedUser 让用户可以查看实际发给 LLM 的 prompt。

per-node 覆盖

```
interface NodeOverride {
  llmConfigName?: string;     // 引用 llm-configs.json 中的配置名
  temperature?: number;
  maxTokens?: number;
  topP?: number;
  frequencyPenalty?: number;
  presencePenalty?: number;
}

interface WorkspacePreferences {
  nodeOverrides?: Record<string, NodeOverride>;
}
```

Workflow 导入/导出

Workflow 文件用连续编号 `"0","1","2",...` 作为 nodeId，实例化为 workspace 时替换为 nanoid。`convert.ts` 提供双向转换：

- `importWorkflow(workflow)` → 连续编号 → nanoid，返回 NodeDef[]
- `exportWorkflow(nodes, name)` → nanoid → 连续编号，返回 Workflow

所有 TextBlock 中的 ref nodeId 也同步重映射。

类型与 Schema 的关系

types.ts 手写 TypeScript 类型，schema.ts 手写 Zod schema（运行时校验），两份各自维护，不用 `z.infer`。

理由：types.ts 的类型用于编译时类型检查，schema.ts 的 Zod schema 用于运行时校验外部输入（JSON 文件、用户配置）。两者的关注点不同—前者追求表达力，后者追求校验严格性。

记忆架构

设计哲学

约束查询是核心能力

记忆模块的底层能力是**约束查询**：给出新内容，判断它与已有章节有没有冲突、冲突在哪里。

关键洞察：**约束查询和上下文注入是同一个能力的两面。** Agent 拿节点 prompt + 上游输出作为锚点去 memory 做约束查询，查到的相关约束就是该注入的上下文。不存在独立的“检索阶段”和“生成阶段”。

Memory 是 manuscripts 的有损缓存

Memory 文件不是独立的知识库，而是 manuscripts（定稿章节）的**有损缓存**。类比 coding agent：代码库是 memory，grep 是检索。对于 FlowCabal：`manuscripts/` 是完整信息源，memory 文件是为了避免每次都加载全部原文的缓存层。

不用 RAG

RAG 的 embedding 召回基于语义相似性，但小说中语义相似的段落可能有几十上百种，召回无法区分哪个是当前需要的。Memory 的跳转链接是**因果关系驱动**的检索，比语义相似性更准确。

种子文件

init 时只创建有**真实约束力**的文件—即 Agent 无法从 manuscripts 自动衍生的东西：

文件/目录	约束域	内容
`index.md`	导航	L0 索引，Agent 的导航入口。自动生成，列出所有 memory 文件及其一行摘要
`voice.md`	叙事一致性	POV、文体、句法模式。需要句法级正例和反例，不能只是抽象标签。含类型叙事约束
`characters/`	角色一致性	一角色一文件。生成性事实：背景因果→性格→动机→关系。写因果链，不写特征列表
`world/`	设定一致性	一概念一文件。世界硬规则、体系原理、边界。含类型设定约束（如“硬科幻：无魔法”）
`manuscripts/`	全精度信息源	定稿章节。L2 层，通过跳转链接按需可达

为什么不是更多种子文件

以下文件在设计中被明确排除：

- `premise.md` — 梗概是高层 outline（作者意图，约束力不可靠）；类型拆入 world + voice 各一行；主题内核太抽象，index.md 一句话够了
- `outline.md` — 纯作者意图，最善变，无约束力
- `chronicle.md` — 初始为空，是 Agent 从 manuscripts 按需衍生的缓存，不该是种子
- `threads.md` — 初始为空，已落笔伏笔的义务，Agent 在写作过程中自行创建

设计原则：init 时只播种 Agent 无法从 manuscripts 自动衍生的东西。其余文件在写作过程中由 Agent 自行创建，index.md 维护导航。

生成性事实 vs 派生断言

Memory 文件应写**生成性事实**（intensional），不写**派生断言**（extensional）：

- 好："奥托对卡莲之死的愧疚驱动他所有决策" — 一条生成规则，可以推导出无限多具体行为
- 坏："奥托不会放弃复活卡莲 / 奥托不会信任陌生人 / 奥托说话彬彬有礼" — 试图穷举无限集合

三级上下文加载

级别	内容	何时加载
L0	`index.md` — 导航入口	Agent 每次启动时自动加载（assembler.ts 的 loadL0）
L1	各记忆文件（characters/ 等）	Agent 读 L0 后通过 read_memory 按需加载
L2	`manuscripts/` 原文	通过跳转链接按需可达

文件间通过**稀疏跳转链接**（`→ path/to/file.md`）构成有向图，Agent 按需导航。

Memory 文件格式

每个 memory 文件的第一行格式：`# 标题 — 一句话摘要`。index.md 自动提取各文件的第一行（截断到 80 字符）构建索引。

Memory 文件命名约定：一主题一文件，子目录组织（characters/、world/）。交叉引用格式 `→ path/to/file.md`。

人和 Agent 共同读写

所有记忆文件对人和 Agent 完全开放。用户可以直接用文本编辑器修改 memory 文件，Agent 也可以在写作过程中创建新文件或更新已有文件。

Agent Memory 工具集（定义在 tools.ts）：

工具	说明
`list_memory`	列出所有 memory 文件（.md，不含 index.md）
`read_memory`	读取指定文件（相对路径），返回 { path, content }
`write_memory`	全量覆写指定文件（prompt 提醒 Agent 先 read 再 write）
`delete_memory`	删除指定文件
`update_index`	重新生成 index.md（遍历所有 .md 文件，提取首行）
`query_runtime`	查询运行时状态（节点列表/输出/状态/版本）。条件启用—仅在传入 RuntimeContext 时可用

Runner-Core：增量构建引擎

执行模型

Runner-core 是一个**增量构建系统**，以 node 为粒度缓存。执行流程：

```
Kahn 拓扑排序 → 分层（topoLevels）→ 逐层并行执行：
  对每个节点：
    1. 结构性解析 TextBlock[]（literal 保留，ref→上游输出，agent-inject→占位符）
    2. 计算 resolved prompt 的 SHA-256 hash（仅 literal + ref 部分）
    3. 与 current version 的 promptHash 比对
    4. 匹配 → 跳过执行，emit node:cache-hit
    5. 不匹配 → 执行 agent-inject → 完整解析 → 流式调 LLM → addVersion
```

失效级联

节点 A 的输出变了 → 节点 B 引用了 A（ref）→ B 的 resolved prompt 变了 → hash 不匹配 → B 重跑 → 依此类推。

这个模型的动机是**用户调试工作流的实际行为**：改一个节点的 prompt → 跑一下看效果 → 再改 → 再跑。以 node 为粒度缓存意味着只有受影响的节点需要重跑。

子图执行

用户可以指定 target 节点，`computeSubgraph()` 通过 DFS 递归访问所有依赖，计算最小执行闭包。只有闭包内的节点参与拓扑排序和执行。

Token 估算

`budget.ts` 提供 CJK/英文双计数的 token 估算：

- CJK 字符：1.5 字符/token
- 其他字符：4 字符/token
- 公式：`ceil(cjkCount / 1.5 + otherCount / 4)`

intentionally 不精确，用于执行前的成本预估（ExecutionPlan.estimate）。

State 中心化

v6 的核心变更。`state.ts` 是全部运行时数据的唯一内存模型。

loadState

`loadState(rootDir, workspaceId)` 一次性加载全部数据到内存：

```
const state = await loadState(rootDir, workspaceId);
// 一次 Promise.all:
// - readdir(outputs/) → 并行读全部 <node-id>.json
// - nodes.json
// - preferences.json
// - 构建 deps/reverseDeps 图
```

加载完成后，所有读操作**同步**完成，零 I/O。

同步读 / 异步写

读操作（内存，同步）：

- `getNodes()` — 全部节点定义
- `getNodeOutput(id)` — current version 的 output（或 null）
- `getNodeStatus(id)` — cached / stale / pending
- `getVersions(id)` — 全部版本
- `getCurrentVersion(id)` — current 版本（或 null）
- `getOutputsMap()` — Map<nodeId, output>（全局快照）
- `computeStructuralHash(id)` — 实时计算节点的结构性 hash
- `getSubgraph(targets)` — 子图节点集
- `getPreferences()` — WorkspacePreferences

写操作（内存先更新 → await 磁盘持久化 → emit StateEvent）：
- `addVersion(nodeId, opts)` — 新增版本并设为 current
- `switchVersion(nodeId, versionId)` — 切换 current 标记
- `updateNodes(nodes)` — 更新节点定义（自动重建依赖图）
- `updatePreferences(prefs)` — 更新偏好

写操作的关键语义：**内存先更新**。写操作返回前，同步读就已经能看到新数据。下游节点的 hash 也立即因上游输出变化而失效。

NodeStatus 计算

状态是**导出属性**，不存储，实时计算：

- `"cached"` — 有 current version 且其 promptHash 与当前结构性 hash 匹配
- `"stale"` — 有 current version 但 promptHash 不匹配（上游变了）
- `"pending"` — 没有任何 version

依赖追踪

state 维护两张图：
- `deps: Map<nodeId, Set<depId>>` — 正向依赖（从 TextBlock ref 推导）
- `reverseDeps: Map<nodeId, Set<downstreamId>>` — 反向依赖

`computeAffected(nodeId)` 用 BFS 遍历 reverseDeps，返回 `[nodeId, ...全部下游]`。用于 StateEvent 的 `affected` 字段—告诉前端哪些节点需要刷新。

`updateNodes()` 时自动重建两张图（因为 ref 可能变了）。

Workspace API

Workspace 是 state 之上的薄 API 层。`openWorkspace()` 异步工厂函数，内部调 `loadState()`。

查询（同步）

```
getNodes(): NodeDef[]
getNodeStatus(nodeId): NodeStatus
previewNode(nodeId): PromptPreview    // 解析 refs，未解析的列入 unresolvedRefs
getVersions(nodeId): NodeVersion[]
getCurrentVersion(nodeId): NodeVersion | null
estimateCost(): TokenEstimate          // 只统计 stale/pending 节点
getDashboard(): WorkspaceDashboard     // { targets, subgraph, nodes[] with status }
```

Mutation（异步）

```
setNodeOutput(nodeId, text): Promise<void>    // human-edit 版本
pickVersion(nodeId, versionId): Promise<void>  // 切换版本
addBlock(...): Promise<void>                   // 结构编辑
removeBlock(...): Promise<void>
moveBlock(...): Promise<void>
setNodeOverride(nodeId, override): Promise<void>
```

执行

```
startRun(opts: { mode: RunMode; signal?: AbortSignal }): RunHandle
```

startRun 内部：
1. 从 targets 计算子图（computeSubgraph）
2. 子图节点拓扑分层（topoLevels）

3. 调 run.ts 的 startRun，传入 state + 子图信息

事件与 Agent

```
onChange(listener: (event: StateEvent) => void): () => void
createRuntimeContext(): RuntimeContext  // 从 state 实时读取
```

`createRuntimeContext()` 返回的 RuntimeContext 直接委托 state—Agent 在执行期间调 `query_runtime` 能看到已完成节点的输出。

Run 执行循环

启动与时序

`startRun()` 返回 RunHandle 后，执行循环通过 `queueMicrotask` 延迟启动。这保证调用者有机会先 `subscribe` 再收到第一个事件（`run:planned`）。

执行流程

```
run:planned  →  run:start  →  [level:start → nodes... → level:done/paused]×N  →
run:done
```

每层的节点并行执行。对于单个节点：

1. emit `node:start`
2. 结构性解析 → 计算 promptHash
3. 检查上下文过期（比较 memory/index.md 的 mtime 与 node output 的 mtime）
4. 缓存命中 → emit `node:cache-hit` + `node:done`
5. 缓存未命中 →
   - 收集 agent-inject hints → 对每个 hint 运行 Agent → emit `node:agent-inject`
   - 完整解析（substitue agent-inject 内容）
   - 流式调 LLM → 逐块 emit `node:generating`
   - `state.addVersion()` 保存结果（含 ExecutionTrace）
   - emit `node:done`

LLM 配置解析

`resolveLlmConfig()` 优先级：

1. `preferences.nodeOverrides[nodeId].llmConfigName` → 从 llm-configs 查找对应配置
2. 回退到 llm-configs 中的 `"default"` 配置
3. override 中的参数（temperature 等）覆盖基础配置

RunHandle

```
interface RunHandle {
  subscribe(listener: (event: RunEvent) => void): () => void;
  advance(): Promise<void>;              // step 模式推进到下一层
  abort(): void;                          // 通过 AbortSignal 取消
  done: Promise<RunSummary | null>;       // null if aborted/error
}
```

step 模式

step 模式下，每层执行完后 emit `level:paused`（携带该层结果），等待 `advance()` 调用。暂停期间 state 是内存的，用户编辑（通过 workspace API）立即可见。

RunSummary

```
interface RunSummary {
  totalNodes: number;
  cachedNodes: number;
  generatedNodes: number;
  errorNodes: number;
  durationMs: number;
}
```

缓存二维失效

结构性失效（自动）

promptHash = SHA-256(literal + ref 展开后的文本)。hash 变化 → 自动重跑 → 级联下游。

agent-inject 不参与 hash 计算（用占位符 `[AGENT-INJECT: {hint}]`）—因为 Agent 每次注入的内容可能不同，不应成为失效条件。

上下文过期（预警）

当 project 的 memory/index.md 被修改时（mtime 变化），现存的 agent-inject 缓存可能过期。处理方式：比较 memory/index.md 的 mtime 与 outputs/<node-id>.json 的 mtime，如果 memory 更新则 emit `context:stale-warning`。

用户决定是否对特定节点重新触发 agent-inject。不自动失效—Agent 查询比较贵（需要多次 tool calling），频繁编辑记忆时全量重跑代价太高。

Workspace 生命周期

Workspace 只有两个状态：**存在**或**不存在**。

- 创建：`createWorkspace()` 生成 nanoid，写 meta.json
- 使用：`openWorkspace()` 加载 state，反复调试，增量构建
- 删除：释放全部缓存，不可恢复

不存在“归档”或“关闭”状态。

事件模型

双通道事件系统，均基于 discriminated union，可序列化。

RunEvent（执行生命周期）

由 run.ts emit，通过 `RunHandle.subscribe()` 消费：

```
type RunEvent =
  // Run 生命周期
  | { type: "run:planned"; plan: ExecutionPlan }
  | { type: "run:start" }
  | { type: "run:done"; summary: RunSummary }
  | { type: "run:error"; error: string }
  | { type: "run:aborted" }
  // Level
  | { type: "level:start"; level: number; nodeIds: string[] }
  | { type: "level:done"; level: number; results: LevelNodeResult[] }
  | { type: "level:paused"; nextLevel: number; results: LevelNodeResult[] }
  // Node
  | { type: "node:start"; nodeId: string; label: string }
```

```
  | { type: "node:cache-hit"; nodeId: string; versionId: string }
  | { type: "node:agent-inject"; nodeId: string; hint: string }
  | { type: "node:generating"; nodeId: string; chunk: string }
  | { type: "node:done"; nodeId: string; versionId: string;
      cached: boolean; output: string }
  | { type: "node:error"; nodeId: string; error: string }
  // 上下文预警
  | { type: "context:stale-warning"; nodeId: string; reason: string };
```

关键设计：

- node:done 携带 output — 前端不需要再回查 getNodeOutput
- level:done / level:paused 携带 results: LevelNodeResult[] — 包含每个节点的 output、versionId、cached 状态
- run:planned 携带 ExecutionPlan — 包含分层、总节点数、缓存节点数、token 估算

StateEvent（状态变更）

由 state.ts emit，通过 Workspace.onChange() 消费：

```
type StateEvent =
  | { type: "version:added"; nodeId: string;
      versionId: string; affected: string[] }
  | { type: "version:switched"; nodeId: string;
      versionId: string; affected: string[] }
  | { type: "nodes:changed"; affected: string[] }
  | { type: "targets:changed"; targets: string[] }
  | { type: "preferences:changed"; nodeId: string };
```

affected = [nodeId, ...全部下游]（BFS 遍历 reverseDeps）。前端据此刷新受影响节点的 UI。

StateEvent 在 run 执行和用户手动操作时都会触发：

- run 生成完一个节点 → state.addVersion() → emit version:added
- 用户手动编辑输出 → workspace.setNodeOutput() → emit version:added
- 用户切换版本 → workspace.pickVersion() → emit version:switched

双通道的好处

- **可序列化** — JSON.stringify 直接发 WebSocket
- **可扩展** — 加新事件不改函数签名
- **可回放** — 录下来就是 event[]，测试/调试用
- **关注点分离** — RunEvent 是执行的临时流，StateEvent 是持久的状态变更通知。前端可以只监听 StateEvent 做 UI 更新

Agent 系统

单 Agent + Tool Calling

v4 的三个专门角色简化为**单个 Agent + tool calling**。理由：

1. 上下文组装 = Agent 通过约束查询自主完成（read_memory + L0 导航）
2. 工作流构建 = 用户定义 workflow 模板
3. 事实检查 = 约束查询的自然副产物

当规模增大到需要专门角色时，再分拆。

两种运行模式

runAgent（非流式）
用于 agent-inject 场景—run.ts 遇到 agent-inject 块时调用。Agent 执行 tool calling 循环（read_memory，query_runtime 等），最终返回注入文本。

底层调 Vercel AI SDK 的 `generateText()`。

conversationalAgent（流式）
用于交互式对话—用户与 Agent 讨论故事、管理 memory。返回 AsyncGenerator，逐块 yield 文本。

底层调 Vercel AI SDK 的 `streamText()`。

agent-inject 机制

agent-inject 是 FlowCabal 的核心差异化能力。当 run.ts 遇到 `kind: "agent-inject"` 的 TextBlock 时：

1. 将节点 prompt + 已解析的上游输出作为锚点
2. 使用 SYSTEM_PROMPT_INJECT 作为系统提示（指导 Agent 提取上下文、保持简洁）
3. Agent 读 L0 索引（assembler.ts 的 loadL0 自动注入），按需导航到 L1/L2
4. 执行约束查询：判断当前上下文与已有内容的关联和潜在冲突
5. 查询结果即为注入内容—约束查询和上下文注入是同一个动作
6. 注入结果缓存在 NodeVersion 的 `agentInjects` 字段中

RuntimeContext

Agent 通过 RuntimeContext 接口查询运行时状态：

```
interface RuntimeContext {
  getNodeOutput(nodeId: string): string | null;
  getWorkflowNodes(): NodeDef[];
  getNodeStatus(nodeId: string): NodeStatus;
  getNodeVersions(nodeId: string): NodeVersion[];
}
```

RuntimeContext 从 state.ts 实时构建。每个 level 执行前可重建，保证 Agent 看到前层结果。Agent 通过 `query_runtime` 工具暴露此接口（支持 list / output / status / versions 四种 action）。

四种系统提示词

prompts.ts 定义四种角色的中文系统提示词，共享 MEMORY_CONVENTIONS 段落（文件组织、交叉引用格式、write_memory 全量覆写等约定）：

提示词	用途
SYSTEM_PROMPT_ANALYZE	分析已有章节，提取生成性事实到 memory
SYSTEM_PROMPT_GENERATE	内容生成，遵循约束、保持连续性
SYSTEM_PROMPT_CHAT	交互式对话顾问，讨论故事/管理 memory
SYSTEM_PROMPT_INJECT	agent-inject 专用，提取上下文、输出注入内容、保持简洁

L0 索引在运行时动态注入到系统提示中（${basePrompt}\n\n 当前索引:\n${l0}），不硬编码文件名。

目录结构

全局配置 ~/.config/flowcabal/

```
~/.config/flowcabal/
├── llm-configs.json              # LLM 配置池（Record<name, LlmConfig>，一套
"default"）
├── workflows/                    # Workflow 模板（纯蓝图，用于分享）
│   └── <workflow-id>.json
└── preferences/                  # 用户对模板的个性化配置
    └── <workflow-id>.json        # per-node LLM 覆盖等偏好，跨工作区生效
```

项目本地 <project>/.flowcabal/

```
<project>/.flowcabal/
├── memory/                        # Agent 记忆
│   ├── index.md                  # L0 导航（自动生成：文件列表 + 首行摘要）
│   ├── voice.md                  # 文体约束 + 类型叙事约束
│   ├── characters/               # 角色（一角色一文件，Agent 按需创建）
│   ├── world/                    # 世界观（一概念一文件，Agent 按需创建）
│   └── manuscripts/              # L2 完整信息源（定稿章节）
└── runner-cache/                  # 工作区（按 workspace 隔离，删除即释放）
    └── <workspace-id>/
        ├── meta.json             # WorkspaceMeta { projectId, createdAt }
        ├── nodes.json            # NodeDef[]（workspace 的真实节点来源）
        ├── preferences.json      # WorkspacePreferences { nodeOverrides }
        └── outputs/
            └── <node-id>.json    # NodeVersionFile { versions[], currentId }
```

路径定义见 packages/engine/src/paths.ts。种子文件/目录定义见 MEMORY_SEED_FILES 和 MEMORY_SEED_DIRS 常量。

LLM 集成

Provider 工厂

provider.ts 根据 LlmConfig.provider 选择 Vercel AI SDK 的对应 provider：

provider 值	SDK 包
openai	@ai-sdk/openai
openai-compatible	@ai-sdk/openai（覆盖 baseURL）
anthropic	@ai-sdk/anthropic
google	@ai-sdk/google
mistral	@ai-sdk/mistral
xai	@ai-sdk/xai

cohere	@ai-sdk/cohere

openai-compatible 复用 @ai-sdk/openai，只是覆盖 baseURL。DeepSeek 等国产模型走这个路径。

生成接口

- generate(config, system, prompt, abortSignal?) — 非流式，返回完整文本。用于 Agent 的 tool calling 循环
- createStream(config, system, prompt, abortSignal?) — 流式，返回 StreamTextResult。用于创作生成，run.ts 通过 textStream 逐块读取并 emit node:generating

两者都将 LlmConfig 中的全部参数（temperature, maxTokens, topP, penalties, providerOptions）传给 SDK。

前端适配

当前 CLI-only，但设计兼容未来 Web UI。state 中心化使以下保证成立：

维度	设计
同步查询	所有查询方法（getDashboard, getNodeStatus, previewNode 等）零 I/O，前端可高频调用
事件驱动	StateEvent（onChange）提供 mutation 事件流，前端监听即可实时更新 UI，无需轮询
纯库	engine 不 console.log、不假设 terminal，副作用通过事件上报
可中断	RunHandle 接受 AbortSignal + done Promise，预留暂停/取消/重跑
可序列化	所有类型保持 JSON-serializable，事件可直接 JSON.stringify 发 WebSocket
API 统一	CLI 和 Web 前端调同一个 openWorkspace() → Workspace 接口

架构决策记录

为什么不用 RAG

问题	分析
语义歧义	小说中语义相似的段落可能有几十上百种，embedding 召回无法区分哪个是当前需要的
因果盲区	伏笔和回收在语义空间里往往很远，余弦相似度捕捉不到因果链
事实幻觉	召回错误段落导致的不是普通幻觉，而是事实性幻觉—基于错误上下文的逻辑自洽输出

基础设施	不需要 embedding 模型、向量数据库、索引构建

Agent 驱动的导航（L0 索引 + 跳转链接）比语义相似性更准确，因为链接是因果关系驱动的。

为什么纯文件系统

维度	分析
可读性	Markdown 文件可直接阅读、编辑—人和 Agent 共同读写的前提
Git 友好	纯文本文件天然支持 diff、merge、history
足够	50 个小文件的 CRUD 不需要数据库
零依赖	不需要 SQLite、LevelDB 或任何外部存储

为什么 Vercel AI SDK

维度	分析
统一接口	一套代码支持 OpenAI / Anthropic / Google / 兼容 API
Tool calling	内置 Zod schema → JSON Schema 转换，自动处理多步 tool calling 循环
流式输出	`streamText` 提供开箱即用的 AsyncIterable
TypeScript 原生	类型安全，与项目技术栈一致

为什么增量构建而非全量执行

用户调试工作流的实际行为是：改一个节点 → 跑一下 → 再改 → 再跑。全量执行意味着每次修改都重跑所有节点（浪费 token + 时间）。以 node 为粒度的增量构建只重跑受影响的节点，SHA-256 hash 比对 + reverseDeps BFS 实现自动级联失效。

为什么 agent-inject 只预警不自动失效

agent-inject 的 Agent 查询比较贵（需要多次 tool calling）。自动失效意味着每次 memory 变更都要重新执行所有 agent-inject—频繁编辑记忆时代价太高。mtime 比对 + `context:stale-warning` 事件让用户自主决定何时重新触发，平衡正确性和成本。

为什么 State 中心化

v5 的 workspace 每次查询走磁盘（10 节点的 getDashboard = 20+ 次 readFile），对 GUI 实时刷新不可接受。

State 中心化的代价：启动时一次性加载全部 version files。对于典型 workspace（10-50 节点，每个 <10 个版本），总数据量 <1MB，加载 <50ms。在可预见的规模内完全可行。

额外好处：run.ts 和 workspace.ts 共享同一份内存数据，Agent 执行期间能看到实时输出—这是无状态设计无法做到的。

为什么 llm/ 在顶层而非 runner-core 下

runner-core 和 agent 都调 LLM。如果 llm/ 放在 runner-core 下，agent 就要反向依赖 runner-core，而 runner-core 遇到 agent-inject 时要调 agent → 循环依赖。所以 llm/ 跟 types/schema/paths 一样是共享基础设施。

为什么 types.ts 和 schema.ts 分开维护

types.ts 的 TypeScript 类型用于编译时类型检查，追求表达力；schema.ts 的 Zod schema 用于运行时校验外部输入（JSON 文件、用户配置），追求校验严格性（如 `z.number().min(0).max(2)`）。两者的关注点不同，强行用 `z.infer` 统一会让双方都变得别扭。

实施路线

Phase 1：Headless Engine + CLI（已完成）

- Monorepo 骨架（engine + cli）
- 类型系统 + Zod schema（types.ts / schema.ts 双轨维护）
- 路径注册表 + ID 生成（paths.ts / id.ts）
- LLM provider 工厂 + generate/stream（llm/）
- Memory 种子初始化 + CRUD + index 自动生成（agent/memory.ts）
- Agent tool-use loop + 四种系统提示词 + RuntimeContext（agent/）
- DAG 拓扑排序 + 子图计算 + Workflow 导入导出（runner-core/workflow.ts + convert.ts）
- TextBlock 解析 + SHA-256 prompt hash（runner-core/resolve.ts）
- State 中心化（内存模型 + 同步查询 + 依赖追踪 + 双通道事件）
- 增量构建执行器（run.ts）+ 多版本缓存 + step 模式
- CLI 命令（init, add-chapter, status, generate, store）

Phase 2：可视化 DAG 编辑器（当前）

在 engine 稳定后开发 Web 前端：

- 可视化工作流编辑（拖拽节点、连线/解连）
- 实时执行可视化（StateEvent 驱动，节点状态实时更新）
- Memory 浏览器（查看/编辑记忆文件）
- Workspace 管理（创建、切换、删除）
- 上下文过期预警 UI
- 版本历史浏览与切换

Engine 包作为 Web 前端的后端，通过直接 import 或 API 层连接。

Phase 3：高级能力（未来）

视实际使用情况决定是否需要：

- 多 Agent 角色分拆（如果单 Agent 不够用）
- 策展管线（自动一致性检查）
- 导入/导出（与其他写作工具互通）